

B 1.1.3 Applying computational thinking



B 1.1.3 Explain how applying computational thinking to fundamental concepts is used to approach and solve problems in computer science.

The four concepts of computational thinking are not abstract ideas, they are active tools applied repeatedly when building software, analysing data, and securing systems. This page explains how and why each concept is used in five core areas of computer science, and what that application looks like in practice.

Last updated: 27 May 2026

Key Questions

- How and why is each computational thinking concept applied in software development, data analysis, and network security?
- Why is it insufficient to apply only one concept when solving a real computer science problem?
- What does it mean to explain the application of computational thinking, and how does that differ from simply identifying which concept is being used?

IB Command Terms

Explain

Give a detailed account, including reasons or causes.

Naming the concept is not enough. You must state what is done, why it is done that way, and what the result is. For each concept in a given domain: what specifically is decomposed, abstracted, or designed, and why does applying that concept help solve the problem?

Exam Technique

A weak answer names concepts: “Pattern recognition is used to detect spam.” A strong answer explains the mechanism: “Pattern recognition allows the system to identify recurring features common to known spam, such as specific keywords and sender behaviour, so the same detection logic applies to new, unseen emails without retraining.” The difference is the explanation of how and why, not just what.

Click the boxes below to find out more:

Software Development

How CT concepts structure the process of building large software systems

Decomposition

A large software system cannot be understood or built as a single unit. Decomposition breaks it into independent modules — for example, a school management system might separate user authentication, timetabling, grade recording, and reporting. Each module can be developed, tested, and debugged independently, allowing multiple developers to work in parallel without interfering with each other.

Pattern recognition

During development, programmers frequently encounter the same problem in different parts of a system, validating user input, or handling database errors. Recognising these as the same pattern means one well-tested solution can be written and reused. This is formalised as design patterns: documented solutions to recurring software design problems that can be applied without solving the problem from scratch each time.

Abstraction

An API (Application Programming Interface) is a practical example of abstraction. When a developer calls a function to send an email, they specify the recipient and content; they do not need to understand how the email is

Algorithmic design

Every feature is implemented as an algorithm. A login feature requires a precise sequence:

Retrieve the stored hash for that username →
hash the input password → compare the two

routed through servers and protocols. The complexity is hidden behind a simple interface, making systems easier to build and maintain without requiring every developer to understand every component.

hashes → if they match, create a session token → if not, increment a failed-attempt counter and return an error.

The order matters; an algorithm that checks the password before retrieving the hash would always fail.

Worked example: e-commerce checkout

Decomposition

The checkout process is separated into basket management, address validation, payment processing, and order confirmation. Each can be updated independently, and for an order that hasn't been fulfilled, the payment module can be replaced without affecting basket management.

Pattern recognition

Basket management, order history, and saved addresses all require the same pattern: fetch a user ID, query a database, return a list. One reusable data-access function is written rather than three separate implementations.

Abstraction

Payment is handled by calling a third-party payment API. The developer specifies amount and card token. The encryption, bank communication, and fraud checks are abstracted away behind the API call.

Algorithmic design

The payment algorithm follows a precise sequence:

Validate basket total → charge payment method → if successful, decrement stock and write order to database → send confirmation email → redirect user.

If any step fails, the algorithm rolls back previous steps to avoid charging without fulfilling the order.

How CT concepts are used to extract meaning from large datasets

Decomposition

A dataset containing millions of records cannot be analysed as a whole. Decomposition divides the analysis into focused sub-tasks: cleaning missing values, filtering to a relevant time period, grouping by category, and calculating summary statistics. Each sub-task produces an output that feeds into the next, making the overall analysis manageable and reproducible.

Pattern recognition

The primary purpose of data analysis is to find patterns not visible in raw data. A pattern might be a trend over time (sales rising each December), a correlation (higher temperature linked to higher ice cream sales), or an anomaly (one account making 500 transactions per hour when the average is 3). Recognising these patterns allows predictions and decisions that would not be possible from inspecting individual records.

Abstraction

Raw data is rarely useful in its original form. A table of 10 million transaction records is too complex to interpret directly. Abstraction reduces this to a meaningful representation: a chart showing monthly revenue trends, or a single figure showing average order value. The same dataset may be abstracted differently depending on the question being asked; the abstraction retains only what matters for that decision.

Algorithmic design

Each stage of a data analysis pipeline is an algorithm. Data cleaning follows a defined sequence:

Identify null values → apply a fill strategy → check for duplicate records → standardise formats.

These steps must occur in this order: standardising formats before removing duplicates may reveal previously hidden duplicates that were masked by inconsistent formatting.

Worked example: school attendance analysis

Decomposition	The analysis is divided into: import raw data → clean and validate records → calculate per-student attendance percentages → identify students below 85% → generate a report for the pastoral team.
Pattern recognition	Analysis reveals that absences cluster on Mondays and Fridays, and that students with low attendance share the same tutor group, suggesting a systemic issue rather than individual cases.
Abstraction	50,000 individual registration records are reduced to a summary table showing each student's name, total sessions, days absent, and attendance percentage, the only figures the pastoral team needs to act on.
Algorithmic design	For each student: Count total sessions → subtract absences → divide by total → multiply by 100 → round to one decimal place → flag if below 85. The algorithm is identical for every student, making it repeatable and auditable.

Network security

How CT concepts are used to analyse, protect, and monitor networked systems

Decomposition A network is too complex to secure as a single unit. Security engineers decompose it into components, client devices, routers, firewalls, servers, and the connections between them, and analyse each independently. This allows vulnerabilities to be located precisely: a misconfigured firewall	Pattern recognition Most attacks leave detectable patterns in network traffic. A port scan produces a pattern of rapid connection attempts statistically different from normal usage. A denial-of-service attack produces unusually high request volume from many sources simultaneously. Intrusion detection systems
--	---

rule is a different problem from an unpatched server. Decomposition also enables defence in depth, applying independent security measures at each layer so that failure at one does not compromise the whole system.	(IDS) are built on this principle: they continuously compare live traffic against known attack signatures, flagging traffic that matches a recognised threat pattern.
Abstraction Security engineers use models to reason about a network without examining every individual packet. A network diagram abstracts the physical infrastructure into a logical representation of nodes and connections. A threat model abstracts the network further, representing only the assets, trust boundaries, and potential attack paths relevant to a security assessment, allowing engineers to identify risks without being overwhelmed by irrelevant detail.	Algorithmic design Security processes are implemented as precise algorithms. Authentication follows a defined sequence: Receive credentials → hash the password → compare to stored hash → if matched, issue a session token with an expiry time → log the attempt. The precision is critical; a single implementation error in an authentication algorithm can expose an entire system.

Worked example — intrusion detection system

Decomposition	The IDS is decomposed into: capture network traffic → parse packet headers → classify each packet by protocol and destination → compare against rule set → generate alert if a rule is matched → log all events. Each component can be updated independently, and new detection rules can be added without changing the capture or logging systems.
Pattern recognition	The system identifies a pattern: 1,200 failed SSH login attempts from the same IP address within 60 seconds. This matches the signature of a brute-force attack. The same username being targeted repeatedly is a further confirming pattern.

Abstraction	The security analyst does not examine all 1,200 individual packets. The IDS abstracts this to a single alert: 'Brute-force attempt detected — source IP 203.0.113.42 — target: SSH port 22 — 1,247 attempts in 61 seconds.' The analyst acts on the abstraction, not the raw data.
Algorithmic design	The response algorithm executes automatically: If failed login attempts from a single IP exceed 100 in 60 seconds → block the IP at the firewall → send alert to the security team → log the block with timestamp → review after 24 hours. The algorithm is deterministic, it produces the same response to the same input every time.

Database design

How CT concepts are used to structure data so that it can be stored, retrieved, and managed efficiently

Decomposition A real-world system involves many different types of entity. For a school library that might include students, books, loans, and staff. Storing all of this in a single table would create unmanageable data and introduce errors, a book title repeated in every loan record wastes space and creates inconsistency if it changes. Decomposition separates the data into individual tables, one per entity type. Relationships between tables, defined using keys, reconstruct the connections between entities when a query requires it.	Pattern recognition During database design, a recurring pattern signals a structural problem: if the same data value appears in multiple rows, a teacher name repeated in every timetable entry, the data contains redundancy. Recognising this pattern triggers normalisation: the repeated data is moved to its own table and replaced with a reference key. The analytical skill involved is recognising functional dependency, identifying when one value in a row determines another, which drives the decision about how to separate tables.
--	--

Abstraction

A database schema is an abstraction of the real world. It represents real entities as tables with defined attributes, retaining only the fields relevant to the system's purpose. A library database representing a student abstracts away their address, medical information, and timetable, retaining only `student_id`, `name`, and `year_group`. The schema also abstracts the physical storage of data, the application interacts with named tables and fields, not with disk sectors, memory addresses, or file structures.

Algorithmic design

Every interaction with a database is executed as an algorithm. A SQL query is a structured sequence of instructions:

SELECT which columns to return → FROM which table → JOIN related tables on a matching key → WHERE specified conditions are met → ORDER BY a column → LIMIT to a number of rows.

The database engine applies these clauses in a defined execution order regardless of how they are written. An incorrect or missing clause produces incorrect, empty, or misleading results.

Worked example — school library database

Decomposition

The library system is decomposed into three tables: Students (`student_id`, `name`, `year_group`), Books (`book_id`, `title`, `author`, `isbn`), and Loans (`loan_id`, `student_id`, `book_id`, `loan_date`, `return_date`). Each table manages one entity, the Loans table connects students to books without duplicating their details in either direction.

Pattern recognition

The designer notices that every loan record would need to store the student full name and book title in full. This repeating pattern signals redundancy, if a book title changes, every loan record would need updating. Normalisation replaces these with `student_id` and `book_id` reference keys, storing the name and title once each in their respective tables.

Abstraction

The Students table abstracts each student to three attributes relevant to library management: `student_id`, `name`, and `year_group`. The student address, photograph, and medical information held in other school systems are irrelevant here and are excluded from this abstraction entirely.

Algorithmic design

Retrieving all overdue loans requires a precise query:

```
SELECT student name, book title, loan date →  
FROM Loans JOIN Students on student_id JOIN  
Books on book_id → WHERE return_date IS  
NULL AND loan_date < today minus 14 days →  
ORDER BY loan_date ascending.
```

Omitting the JOIN would return IDs without readable names; omitting the WHERE condition would return all active loans, not just overdue ones.

Machine learning

How CT concepts underpin systems that learn to recognise patterns from data automatically

WHAT IS MACHINE LEARNING?

Machine learning (ML) is a branch of computer science in which a system learns to perform a task by identifying patterns in training data, rather than by following explicitly programmed rules. A spam filter built with explicit rules lists forbidden words and blocks them; an ML spam filter learns from thousands of labelled examples what spam looks like, and generalises that learned knowledge to classify new emails it has never seen.

Decomposition

Building an ML system involves multiple distinct stages, each a separate engineering problem. The pipeline is decomposed into:

- (1) data collection, gathering labelled examples;
- (2) preprocessing, cleaning and formatting the data;

Pattern recognition

Pattern recognition is not only applied by human designers, it is the core mechanism of the ML system itself. An image classifier learns to recognise the pattern of pixel values that correspond to a cat versus a dog. A fraud detection model learns the statistical pattern of transaction attributes, amount, location, time of day, merchant type, that characterise fraudulent activity. The training algorithm adjusts the model's internal parameters until it reliably distinguishes these patterns in data it

(3) feature selection — identifying which attributes of the data are useful predictors;

(4) model training, adjusting model parameters using the training examples;

(5) evaluation, testing the trained model on unseen data;

(6) deployment, integrating the model into a live system. Failure at any one stage requires returning to that stage rather than restarting the whole process.

has not previously encountered. The system generalises from examples; it does not memorise them.

Abstraction

A trained ML model is an abstraction of the real-world relationship it was trained to represent. An image classifier does not store its training images, it stores a set of learned numerical parameters (weights) that encode the statistical relationships between pixel patterns and class labels. These weights are a compact abstraction derived from data but far smaller than the data itself. Feature engineering, deciding which attributes of raw data to include as inputs, is itself an act of abstraction, selecting the signals that predict the outcome and discarding irrelevant detail.

Algorithmic design

Training an ML model follows a precise iterative algorithm. In gradient descent, a common training approach, the process is:

Initialise model parameters randomly → pass a batch of training examples through the model → compute the prediction error (loss) → calculate how each parameter contributed to the error → adjust each parameter slightly in the direction that reduces error → repeat until the loss is minimised or a stopping condition is met.

This algorithm executes millions of times during training. Evaluation also follows a defined algorithm:

Run the model on held-out test data → compare predictions to known labels → compute accuracy and other performance metrics.

Worked example — email spam classifier

Decomposition	The ML pipeline is decomposed into: collect labelled emails (spam / not spam) → preprocess text (remove punctuation, convert to lowercase, tokenise words) → extract features (word frequencies, link count, sender reputation score) → train a classifier on the labelled examples → evaluate accuracy on a held-out test set → deploy to filter incoming email in real time.
Pattern recognition	The training algorithm analyses thousands of labelled spam emails and identifies statistical patterns: certain words appear far more frequently in spam; spam emails typically contain more links per word; sender domains follow recognisable patterns. The trained model encodes these patterns as weights, allowing it to classify new emails by matching their features against the learned patterns, without any explicit list of forbidden words.
Abstraction	Each email is not processed as raw text, it is abstracted to a feature vector: a list of numerical values representing word frequencies, link count, sender reputation score, and similar measurable attributes. The model operates entirely on these feature vectors. This abstraction reduces an email of any length and format to a fixed-size set of numbers, making classification computationally efficient and the model generalisable across any email.
Algorithmic design	Training follows a defined algorithm: For each batch of labelled emails, pass feature vectors through the model → compute the loss between predicted and actual labels → backpropagate the error to calculate how each parameter contributed → update weights by a small amount in the direction that reduces error → repeat across all batches → stop when validation accuracy stabilises. Classification then applies a threshold algorithm: if the model's spam probability output exceeds 0.85, route to spam folder; otherwise deliver to inbox.

Exam Technique

What an “explain” answer requires

An exam question might read: 'Explain how computational thinking is applied in software development. [6 marks]' A six-mark response needs to address multiple concepts with reasoning. For each concept:

1. Name and define what is done in this domain.
2. Explain the mechanism. Why does applying this concept help?
3. Give a concrete example specific to the domain.

Two concepts explained fully will score higher than four concepts named without explanation.

Model answer extract

Decomposition is used to divide a large software project into smaller, independent modules. *[concept named + what is done]* This is necessary because the full system is too complex to design, build, or test as a single unit. By isolating modules, developers can work on each independently and identify errors without affecting the rest of the system. *[why / mechanism]*

For example, a banking application might be decomposed into separate modules for account management, transaction processing, and fraud detection. *[concrete example]*

Summary

Concept	Why it is applied	Example across domains
Decomposition	Complex problems cannot be solved as a whole; breaking them into independent sub-problems makes each part solvable and testable.	<i>Software modules; analysis pipeline stages; IDS components</i>

Concept	Why it is applied	Example across domains
Pattern recognition	Recurring structures allow solutions to be generalised — the same logic applies to new instances without re-solving the problem.	<i>Design patterns in code; anomaly detection in data; attack signatures in traffic</i>
Abstraction	Retaining only relevant detail allows engineers to reason about a system without being overwhelmed by complexity.	<i>APIs hiding implementation; charts summarising data; network threat models</i>
Algorithmic design	Computers require exact, unambiguous instructions; algorithmic design produces a precise, repeatable sequence that gives a correct result for any valid input.	<i>Login sequence; data cleaning pipeline; IDS response procedure</i>

B 1.1.3 Quizlet

Test your knowledge with this Quizlet



Ready to play?

Match all the terms with their definitions as fast as you can. Avoid wrong matches, they add extra time!

Start game

[View this flashcard set](#)

Choose a Study Mode ▾

Knowledge Check: Quiz

Test your understanding of the core concepts.

1

Fill in the blanks with the appropriate computational thinking concept (decomposition, pattern recognition, abstraction, algorithmic design):

pattern recognition

algorithmic design

abstraction

pattern recognition

pattern recognition

abstraction

decomposition

pattern recognition

algorithmic design

decomposition

A programmer uses **[decomposition]** to break down a complex software project into smaller modules.

A data analyst uses **[pattern recognition]** to identify trends and anomalies in a large dataset.

A machine learning engineer uses **[abstraction]** to create a model that predicts customer churn.

A database administrator uses **[decomposition]** to organize data into tables and relationships.

A network security specialist uses **[pattern recognition]** to detect malicious activity on a network.

A student uses **[algorithmic design]** to plan a study schedule for their final exams.

A chef uses **[algorithmic design]** to create a recipe for a new dish.

A musician uses **[pattern recognition]** to compose a song with repeating melodies and harmonies.

An artist uses **[abstraction]** to create a simplified representation of a landscape.

A doctor uses **[pattern recognition]** to diagnose a patient based on their symptoms and medical history.

being blocked and legal action being taken against you.

 **Report a problem**